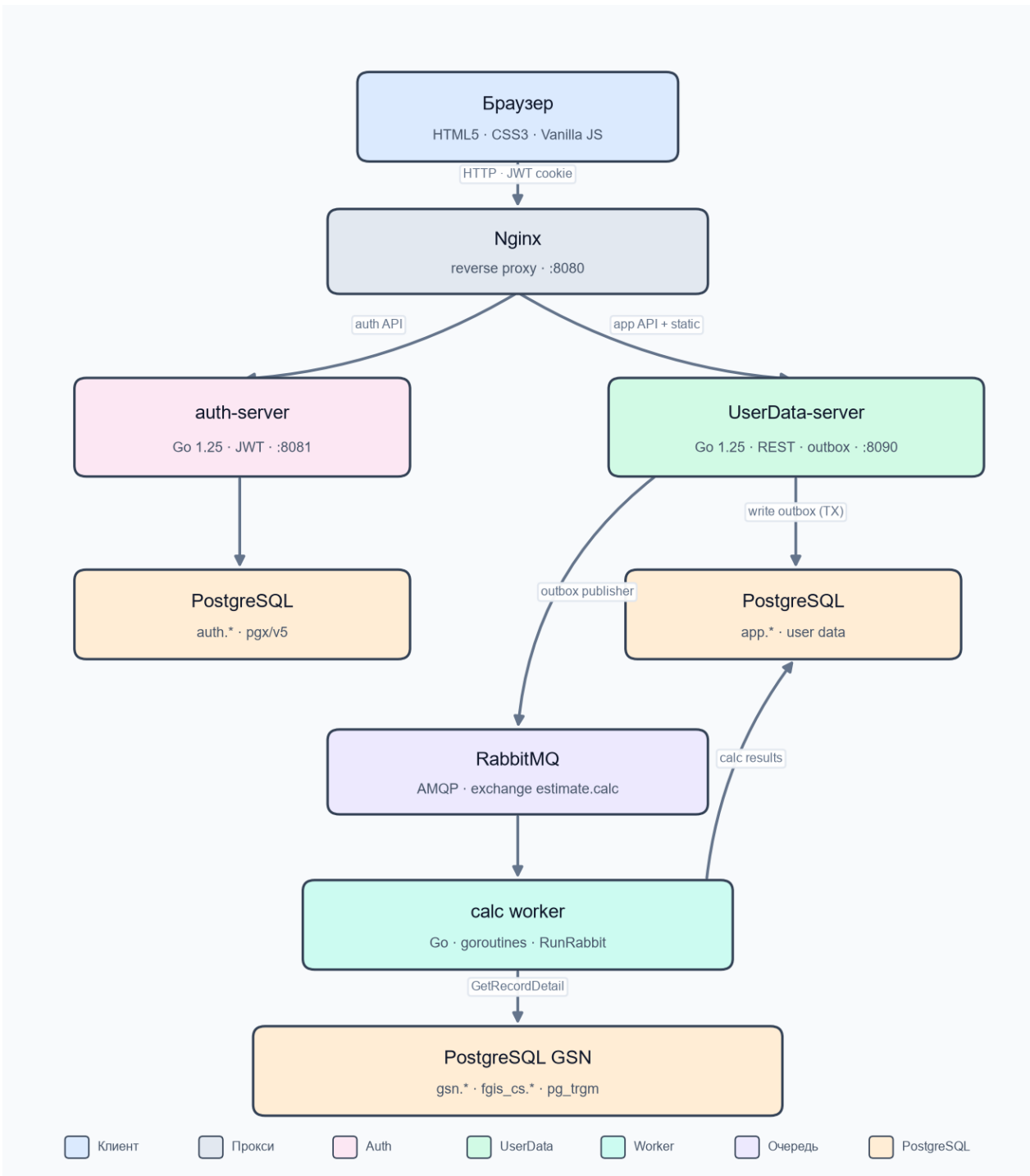


Архитектура проекта NAV SaaS MVP

Версия 2026-06-30

SaaS-платформа для автоматизации выпуска смет (ГСН-2022, ФГИС ЦС). Мультисервисный контур: nginx (единая точка входа) -> auth-сервис + UserData-server + calc worker.

Схема приложения



Клиент: HTML5, CSS3, Vanilla JavaScript, sessionStorage

Прокси: Nginx (deploy/nginx.conf)

Серверы: Go 1.25, net/http, pgx/v5 (PostgreSQL)

Auth: JWT HMAC-SHA256, HTTP-only cookie

Очередь: RabbitMQ (AMQP), transactional outbox

Сборка и запуск: go build, run.bat (Windows)

Общая схема

Запуск одной командой run.bat поднимает четыре компонента. Публичный вход — nginx на :8080 (UI и маршрутизация). Auth API проксируется на :8081, бизнес-API и статика — на UserData-server :8090. Calc worker работает в фоне и не использует JWT.

Клиенты: web/ (основной UI), web/admin.html (админка)

Nginx :8080 -> auth :8081 | app :8090

Процессы: nav-auth-server, UserData-server (nav-server.exe), nav-calc-worker, nginx

Хранилища: PostgreSQL auth.*, app.*, gsn.*/fgis_cs.*; RabbitMQ; data/app.json (legacy)

Процессы (run.bat)

- Nginx (:8080) — единая точка входа; deploy/nginx.conf
- Auth service (:8081) — login, users, companies, licenses; cmd/auth_server
- UserData-server (:8090) — бизнес-API, статика, outbox publisher, healthz; cmd/server
- Calc worker (фон) — потребитель очереди; пишет calc_json/calc_status в app.*

Backend (Go)

Несколько процессов с общими пакетами internal/. App-сервер: backend/cmd/server/main.go.

Auth: backend/cmd/auth_server.

backend/

- cmd/server/ — app API + outbox publisher
- cmd/auth_server/ — auth API (:8081)
- cmd/calc_worker/ — расчёт позиций (Rabbit / legacy DB)
- cmd/migrate_auth/ — импорт users из JSON в auth.*
- internal/api/ — маршруты app, healthz, queue-stats
- internal/authapi/ — handlers auth API
- internal/auth/ — JWT verify (app), пароли (auth)
- internal/authstore/ — PG: auth.companies, auth.users, licenses
- internal/store/ — сметы,стройки, outbox, очередь DB
- internal/gsn/ — справочник ГСН-2022 и ФГИС
- internal/calcworker/ — worker, Rabbit consumer, Manager
- internal/outbox/ — publisher outbox -> RabbitMQ
- internal/queue/ — инспекция очередей RabbitMQ
- internal/presence/ — блокировки смет, лиценз. сессии (in-memory)

Auth-контур

- users/companies/licenses — только PostgreSQL auth.* (db/auth_schema.sql)
- JWT в cookie; claims: name, authorized; app проверяет доступ по claims
- APP_AUTH_DATABASE_URL обязателен; APP_JWT_SECRET одинаковый у auth и app
- Nginx маршрутизирует /api/auth/*, /api/me, /api/users, /api/companies -> :8081
- Go reverse проху в app удалён; auth не участвует в расчёте смет

App API (internal/api/server.go)

- Стройки, объекты, сметы, GSN, settings, estimate-locks, license-sessions
- GET /api/healthz — состояние очереди, publisher/consumer, outbox, DLQ
- GET /api/admin/queue-stats — мониторинг RabbitMQ (только админы)
- GET /api/estimates/{id}/calc-status — polling статуса расчёта
- Статика web/; админка /admin

Хранилище (internal/store)

Предметные данные — PostgreSQL app_* при APP_DATABASE_URL:

- app_constructions, app_construction_objects, app_estimates, app_estimate_lines
 - app_settings, outbox_events, calc_message_receipts (идемпотентность)
 - estimate_calc_jobs — только при APP_QUEUE_MODE=db (legacy fallback)
- data/app.json — legacy snapshot настроек (если нет PG); учётки вынесены в auth.*.

Модуль ГСН (internal/gsn)

Отдельное подключение APP_GSN_DATABASE_URL. Схема — db/gsn_schema.sql:

- gsn.supplements, gsn.hierarchy, gsn.base_info — иерархия ГСН-2022
- fgis_cs.* — наборы и строки ФГИС ЦС (цены по районам)
- Импорт — CLI import_regions, import_resource_codifier, import_fgis_cs

Presence (internal/presence)

- EstimateLocks — эксклюзивная блокировка сметы (TTL ~90 с)
- LicenseSessions — учёт активных лицензий по подразделам ГСН (in-memory)

Асинхронный расчёт смет

Источник истины — текстовая строка (raw_text). Таблица — проекция. Основной транспорт — RabbitMQ (APP_QUEUE_MODE=rabbit).

1. UI -> PUT /api/estimates
2. API -> app_estimate_lines (calc_status=queued) + outbox_events (TX)
3. Outbox publisher -> RabbitMQ exchange estimate.calc
4. Queue estimate.calc.main -> calc_worker (RunRabbit)
5. Worker -> GSN (GetRecordDetail) + app.* (calc_json, calc_status=done)
6. UI polling GET calc-status -> обогащение таблицы

- Идемпотентность: calc_message_receipts (estimate_id, line_id, revision)
- Очереди: main, retry (5s/30s/120s), DLQ; ops: purge/replay DLQ
- Consumer heartbeat в app_settings; healthz: pipelineReady, releaseReady
- Legacy: APP_QUEUE_MODE=db -> estimate_calc_jobs + FOR UPDATE SKIP LOCKED
- Worker: таймаут позиции 40 с, max 8 goroutine, GSN pool по числу worker-ов
- Парсинг объёма из raw_text — backend (quantity_expr.go, normalizeEstimateItem)

Frontend (web/)

SPA без фреймворка — vanilla JS + HTML + CSS.

- index.html + app.js — основное приложение
- admin.html + admin.js — админ-панель

Разделы UI:

- База -> ГСН-2022 (иерархия, поиск), позиции пользователя
- Стройки -> иерархия Стройка -> Объект -> Смета
- Настройки (число worker-ов расчёта, 1–8)

Админка (/admin): пользователи, лицензии, блокировки смет, мониторинг очереди.

Редактор сметы: текстовый и табличный режимы; до расчёта — шифр + объём; после worker — полная строка из calc_json. Состояние — sessionStorage.

Базы данных

```
APP_AUTH_DATABASE_URL:
  auth.companies, auth.users, auth.company_licenses
APP_DATABASE_URL:
  app_*, outbox_events, calc_message_receipts, estimate_calc_jobs (legacy)
APP_GSN_DATABASE_URL:
  gsn.*, fgis_cs.*
```

Переменные окружения

- APP_ADDR (:8090), APP_WEB_DIR, APP_DATA_PATH
- APP_DATABASE_URL, APP_AUTH_DATABASE_URL, APP_GSN_DATABASE_URL
- APP_JWT_SECRET (общий для auth и app)
- APP_QUEUE_MODE=rabbit, APP_RABBITMQ_URL, APP_RABBITMQ_EXCHANGE
- APP_RABBITMQ_PREFETCH, APP_OUTBOX_PUBLISH_BATCH, APP_OUTBOX_PUBLISH_INTERVAL

Вспомогательные инструменты

- scripts/restart-{auth-server,calc-worker,nginx}.bat
- scripts/{smoke,load}-rabbit-calc.ps1, rabbit-queue-status.bat
- scripts/{purge,replay}-rabbit-dlq.bat, rollback-queue-db.bat
- scripts/import_gsn_*.py, backup-sources.ps1

Текущее состояние

- Реализовано: вынос auth в отдельный сервис + auth.* PG + nginx
 - Реализовано: RabbitMQ + outbox + идемпотентность + мониторинг в админке
 - Реализовано: calc worker отдельно; защита от зависания (таймаут, cap 8)
- Следующие шаги (опционально): оптимизация listRecordResources (N+1), алерты при росте DLQ, Prometheus/Grafana поверх healthz.